

Lecture 2: Inleiding tot Machine Learning

1 Overzicht

In de vorige les legden we de statistische grondslagen van data-analyse uit: kansverdelingen, voorwaardelijke verwachtingswaarden en het cruciale onderscheid tussen associatie en interventie. In deze les bouwen we daarop verder en verkennen we hoe moderne machine learning-methoden in de praktijk werken.

2 Terminologie en learning theory

2.1 Wat is machine learning?

Het vakgebied rondom data-analyse kent een rijke maar soms verwarrende terminologie. Begrippen als *artificiële intelligentie*, *machine learning*, *data science* en *deep learning* worden vaak door elkaar gebruikt.

Een handige manier om de begrippen te situeren is als een stelsel van concentrische cirkels:

- **Artificiële intelligentie (AI)** is het bredere vakgebied dat alle systemen omvat die intelligent gedrag nabootsen — inclusief niet-statistische methoden zoals regelgebaseerde systemen.
- **Machine learning (ML)** is een deelverzameling van AI: systemen die *leren* uit data, zonder voor elk scenario expliciet geprogrammeerd te worden.
- **Deep learning** is een deelverzameling van ML: leren via neurale netwerken met meerdere lagen.
- **Data science** is een bredere term die ook dataverzameling, -verwerking, statistiek en communicatie omvat.

💡 Praktische richtlijn

De term AI wordt soms rondgegooid als een vorm van hedendaags magisch denken; in de realiteit zijn er geen shortcuts voor de belangrijke vragen: *welke statistische aanname maakt dit model, en klopt die aanname voor mijn data?*

In een industriële context betekent machine learning in de eerste plaats een verzameling van **statistische modellen en optimalisatiemethoden**. De term AI wordt dan weer meer gebruikt voor het gebruik van LLMs, bijvoorbeeld als code-assistent.

Leerparadigma's

De drie klassieke leerparadigma's in machine learning zijn:

Supervised learning ("gesuperviseerd leren"): het model leert van *gelabelde* data — paren (x_i, y_i) waarbij x_i de observaties zijn en y_i de gekende uitkomst. Het doel is een functie $\hat{f}$ te leren zodat $y \approx \hat{f}(x)$ voor nieuwe, ongeziene data. Link met de vorige les: het zoeken van structuur in $P(Y | X)$, typisch bepalen van $\mathbf{E}[Y|X]$.

Unsupervised learning ("ongesuperviseerd leren"): het model krijgt enkel invoerdata (x_i) — er zijn geen labels. Het doel is *structuur* te ontdekken in de data: clusters, uitschieters, lage-dimensionale representaties. Link met de vorige les: het zoeken van structuur in $P(X)$

Reinforcement learning ("versterkend leren"): een agent leert optimaal gedrag door te interageren met een omgeving en beloningen of straffen te ontvangen. Link met de vorige les: zoeken van structuur in $P(Y | \text{do}(X))$.

Voorbeelden

- **Supervised learning:** er zijn *labels* die voorspeld moeten worden
 - Voorspel de treksterkte van een rol staal op basis van de procesparameters — het label is de gemeten sterkte.
 - Voorspel levertijden voor een recent geplaatste bestelling — het label is de historisch geobserveerde levertijd.
- **Unsupervised learning:** er zijn *geen* vooraf gedefinieerde labels
 - Vind anomalieën in elektrische signalen van een installatie die wijzen op een naderend defect.
 - Detecteer fraude, bv. aan de self-checkout van een supermarkt.



Note

Gevorderde AI-systemen maken vaak gebruik van een complexe combinatie van verschillende leerparadigma's, bijvoorbeeld een niet gesuperviseerde stap om de structuur in de data te leren kennen, gevolgd door een gesuperviseerde stap en/of reinforcement stap.

Supervised learning: regressie en klassificatie

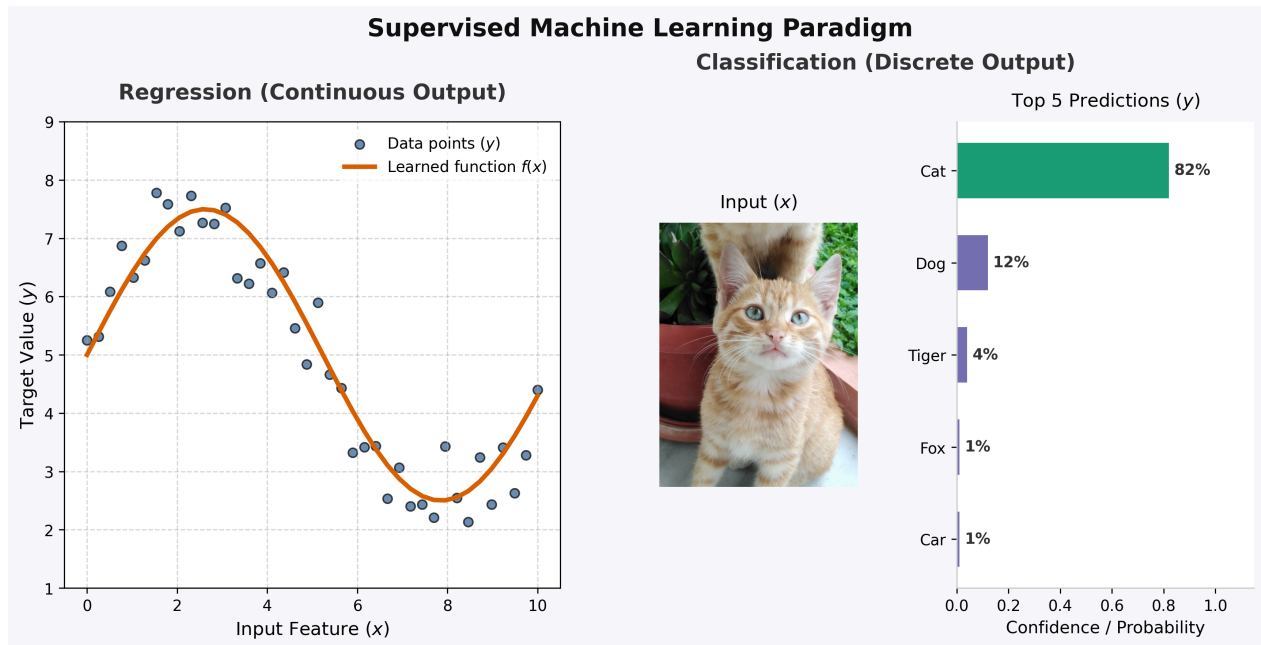
In les 1 hebben we het concept gezien van de voorwaardelijke kans $P(Y | X)$ en de voorwaardelijke verwachtingswaarde $f(x) = \mathbf{E}[Y | X = x]$ onder de naam *regressielijn*.

We gaan ons in deze les concentreren op *supervised learning*, dus op het leren van functies $\hat{f}$ die de functie f zo goed mogelijk benaderen. In praktijk moeten we daarvoor over een gelabelde dataset $D = (\vec{x}_i, y_i)_{i=1}^N$ beschikken en onze schatting zal dan ook afhangen van de data D die we gebruikt hebben om $\hat{f}$ op te bouwen. Men zegt ook dat we een model $\hat{f}$ getraind hebben op de dataset D .

Afhankelijk van het soort variabele dat we proberen te herkennen zijn er twee soorten van supervised learning:

- Als y continu is, spreekt men van **regressie**. Het model leert typisch één waarde voorspellen voor een gegeven input: het model leert $\mathbf{E}[Y | X = x]$. Voorbeelden:

- Voorspellen hoeveel electriciteit een bedrijf nodig heeft, gegeven de productieplanning en weersvoorspelling.
- Voorspellen hoeveel klanten een online aankoop zullen doen 1 week voor kerstmis.
- Als y discreet is, spreekt men van **classificatie**. Het model leert typisch een kansverdeling voorspellen voor een gegeven input: het model leert $\mathbf{P}(Y \mid X)$.
 - Voorspellen of een geïnteresseerde in een woning al dan niet een lening zal kunnen krijgen.
 - Voorspellen of een geproduceerd onderdeel zal worden goedgekeurd, afgekeurd, of afgewaardeerd.



2.2 Loss-functies

Het concept

Hoe weten we of een gegeven machine learning-model $\hat{f}$ een goede voorspelling oplevert? Het antwoord ligt in de **loss-functie** (ook wel *kostfunctie* of het *objectief* genoemd): een wiskundige maatstaf die de afwijking tussen de voorspellingen van het model en de werkelijke waarden kwantificeert.

Formeel: gegeven data (x_i, y_i) voor $i = 1, \dots, N$ en een model $\hat{f}$, definieert de loss-functie $\mathcal{L}$ een getal dat aangeeft *hoe slecht* het model presteert, bijvoorbeeld:

$$\mathcal{L}_D(\hat{f}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, \hat{f}(x_i))$$

Het trainen van een model komt in essentie neer op het **minimaliseren van de loss** over de beschikbare traindata.

⚠ Important

De lossfunctie hangt af van de beschikbare (of gekozen) trainingsdata; daardoor hangt ook de oplossing (die $\hat{f}$ waarvoor de loss minimaal is) af van de beschikbare data!

Veelgebruikte loss-functies voor regressie

Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{f}(x_i))^2$$

De meest klassieke keuze. Door het kwadraat te nemen worden grote afwijkingen zwaarder bestraft dan kleine. Gevoelig voor uitschieters.

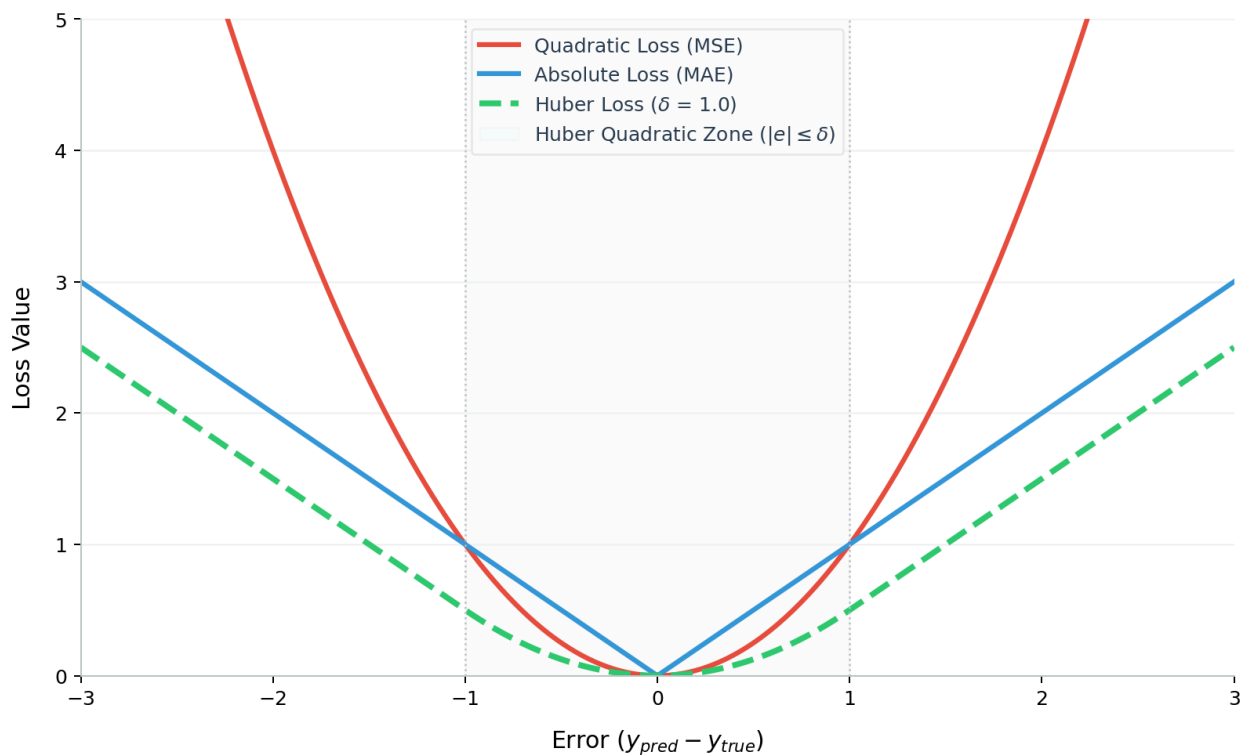
Mean Absolute Error (MAE):

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{f}(x_i)|$$

Robuuster bij uitschieters: grote fouten worden lineair — niet kwadratisch — bestraft.

Huber loss: een combinatie van MSE (voor kleine fouten) en MAE (voor grote fouten), nuttig wanneer de data occasionele uitschieters bevat maar men toch differentieerbaar wil blijven.

Comparison of Common Loss Functions



Veelgebruikte loss-functies voor classificatie

Cross-entropie (ook wel *log-verlies*): de standaardkeuze voor classificatieproblemen. Bestraft het model zwaar wanneer het met hoge zekerheid de verkeerde klasse voorspelt.

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_k y_{i,k} \log \hat{p}_{i,k}$$

waarbij $\hat{p}_{i,k}$ de voorspelde kans is dat observatie i tot klasse k behoort.

⚠ Te onthouden

De keuze van de loss-functie is een **modelleringsbeslissing** die de aandacht van de analist vereist. Ze bepaalt:

1. Wat het model optimaliseert — en dus ook wat het negeert.
2. Hoe gevoelig het model is voor uitschieters.
3. Impliciet welke veronderstelling over de foutenverdeling het model maakt (MSE impliceert Gaussische fouten; MAE impliceert Laplacianse fouten).

Een fout in de keuze van de loss-functie kan leiden tot een model dat technisch gesproken goed traint, maar de verkeerde vraag beantwoordt.

Verband met statistiek

De keuze voor een loss-functie is vaak ingegeven door diepere, statistische overwegingen. We veronderstellen dat het machine learning model een statistische beschrijving is van een proces dat de data kan genereren en we vragen ons af hoe we de plausibiliteit dat de geobserveerde data door het model wordt gegenereerd zo groot modelijk kan worden gemaakt: dit is het maximaliseren van de *likelihood*. Om technische redenen gaat men liever de negatieve log-likelihood minimaliseren en afhankelijk van de set-up is de verliesfunctie die zo geminimaliseerd wordt precies één van de gekende loss-functies.

i Asymmetrische kosten

In een productieomgeving zijn niet alle fouten even erg. Een stuk ten onrechte afkeuren (vals positief) kost productie; een defect stuk doorlaten (vals negatief) kost klachten en mogelijk veiligheid. Een standaard loss-functie behandelt beide fouten gelijkwaardig — maar een *gewogen* loss-functie kan dit corrigeren door aan de ene fout een zwaarder gewicht toe te kennen.

2.3 Trainen van modellen

Wat is een model?

In de vorige sectie spraken we over de loss-functie $\mathcal{L}(\hat{f})$ en over het minimaliseren ervan. Maar wat is $\hat{f}$ precies?

Een **model** is een *klasse van functies*, geparametriseerd door een vector van **gewichten** (ook wel *parameters* of *coëfficiënten* genoemd) die we noteren als θ . We schrijven $\hat{f}_\theta$ om duidelijk te maken dat de gekozen functie volledig bepaald wordt door de keuze van θ .

Voorbeelden:

- Bij **lineaire regressie** zijn de gewichten de coëfficiënten $\theta = (\beta_0, \beta_1, \dots, \beta_p)$. De klasse van functies is de verzameling van alle lineaire functies.
- Bij een **neuraal netwerk** zijn de gewichten de verbindingsterktes tussen alle neuronen — van een paar dozijn tot miljarden getallen.
- Bij een **beslissingsboom** zijn de gewichten de drempelwaarden en knooppunten van de boom.

Trainen = gewichten optimaliseren

Trainen van een model is niets anders dan het **zoeken naar de gewichten $\hat{\theta}$ die de loss-functie minimaliseren** op de beschikbare traindata:

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\hat{f}_\theta)$$

De zinsnede *op de beschikbare traindata* is hier van groot belang: als we opnieuw zouden trainen op een andere dataset gaan we een andere $\hat{\theta}$ vinden, een meer nauwkeurige notatie zou daarom zijn:

$$\hat{\theta}_D = \arg \min_{\theta} \mathcal{L}_D(\hat{f}_\theta)$$

Meestal zijn er voor een gegeven modelklasse zeer goede algoritmen beschikbaar die de optimale parameters snel kunnen vinden. Specialist in machine learning zullen deze algoritmes dan ook niet steeds zelf implementeren maar vertrouwen op beschikbare implementaties.

Tip

Moderne taalmodellen zoals GPT, Gemini en Claude zijn parametrische modellen, maar met onwaarschijnlijk veel parameters. De “originele” ChatGPT (GPT-3.5 uit 2022) was een 175B model, wat wil zeggen dat er 175 miljard (billion) te trainen parameters zijn. Latere modellen zoals GPT-4 en Claude 3 zijn naar schatting 1T-2T modellen, dus met 1 biljoen (trillion) parameters. Het open weights model DeepSeek-V3/R1 is een 671B-model.

Parameters versus hyperparameters

Naast het trainen van de parameters moet de analist ook goede keuze maken voor de zogenaamde hyperparameters: een hoger niveau van keuzes waarvan de optimale waarde meestal niet uit het trainingsalgoritme rolt.

Parameters (θ): de gewichten die het trainingsproces zelf optimaliseert. De analist hoeft ze niet handmatig in te stellen.

Hyperparameters: keuzes die de *structuur* of *complexiteit* van het model bepalen — ze worden *niet* door de loss-minimalisatie bepaald, maar moeten vooraf ingesteld worden. Voorbeelden:

- Het aantal termen (graad) in een polynomiale regressie
- Het aantal lagen en neuronen in een neuraal netwerk
- De diepte van een beslissingsboom
- De regularisatiesterkte λ bij ridge-regressie

Het probleem met meer complexiteit

Het lijkt voor de hand liggend om voor een zo complex mogelijk model te kiezen: meer parameters geven het model meer vrijheid om de data te beschrijven, en daardoor kan het de traindata beter fitten. En inderdaad: als we enkel naar de **loss op de traindata** kijken, zal een complexer model het altijd minstens even goed doen als een eenvoudiger model.

Maar dit geeft een vals gevoel van succes. Wat we werkelijk willen weten is hoe goed het model presteert op **nieuwe, ongeziene data**. Een te complex model “memoriseert” de traindata — inclusief de ruis — en verliest daarmee zijn vermogen om te generaliseren. Dit is het fenomeen van **overfitting** dat we verder uitwerken in de sectie over underfitting en overfitting.

De vraag is dus: hoe vind je de juiste hyperparameters en optimale modelcomplexiteit? De meest gebruikte werkwijze bestaat erin dat we het model evalueren op data die niet gebruikt werd voor training, wat men ook wel **cross-validatie** noemt.

2.4 Cross-validatie

Om te meten hoe goed een model generaliseert naar ongeziene data hebben we dus een **onafhankelijke evaluatie** nodig.

Train-test splitsing

De eenvoudigste oplossing is de dataset te verdelen in een **trainset** en een **testset**:

1. Train het model op de trainset.
2. Evalueer de prestatie op de testset (die het model nooit gezien heeft).

Typisch wordt 70–80% van de data gebruikt voor training en 20–30% voor test. De testset simuleert “toekomstige data”. De analist kan dan goede keuzes maken voor de optimale complexiteit van het model door de prestatie op de testset te vergelijken.

Cross-validatie is de standaard techniek om **hyperparameters** te kiezen: modelkeuzes die niet door het trainproces worden bepaald, maar die de complexiteit van het model beïnvloeden.

Voorbeelden:

- Het aantal termen in een polynomiale regressie
- De diepte van een beslissingsboom
- Het aantal neuronen in een neuraal netwerk
- De regularisatiesterkte λ in ridge- of lasso-regressie

Werkwijze: train het model voor verschillende hyperparameter-waarden, evalueer via cross-validatie, en kies de waarde met de beste prestatie op de testset.

! Validatie set

Soms zal je zien dat door herhaaldelijk te trainen tot je de beste prestatie verkregen hebt op de testdata het model effectief overtraind is op de testset: de keuze van hyperparameters is optimaal voor de keuze van train-test split die de analist toevallig gemaakt heeft, maar misschien niet voor andere testsets. Regelmatig lees je daarom de aanbeveling om nog een *derde* dataset bij te houden: de **validatieset**.

Bovendien, om het extra verwarrend te maken gebruiken veel mensen de terminologie validatie- en testset omgekeerd aan wat we hier doen. Meestal blijkt uit de context wat er precies bedoeld wordt.

k -voudige cross-validatie

Je kan ook een stapje verder gaan als je niet wil dat de resultaten afhankelijk zijn van de toevallige keuze van de splitsing: *k*-voudige cross-validatie (*k*-fold cross-validation) pakt dit aan.

1. Verdeel de data in k gelijke delen ("folds").
2. Train het model k keer, waarbij telkens één fold als testset dient en de overige $k - 1$ folds als trainset.
3. Middel de k prestatiemetingen.

Zo krijg je een stabielere schatting van de generalisatieprestatie. Typische waarden zijn $k = 5$ of $k = 10$. Nadeel is dat je het model veel vaker gaat moeten trainen.

2.5 Underfitting en overfitting



Illustratie van underfitting (links), een goede fit (midden) en overfitting (rechts) op dezelfde dataset.

Naarmate de waarde van een hyperparameter verandert van heel klein naar heel groot, zien we vaak dat de prestatie op de testset eerst sterk afneemt en later terug sterk toeneemt: dit is de typisch **U-vormige loss-curve**.

- **Underfitting**: het model is te eenvoudig en mist structuur in de data. Zowel de trainfout als de testfout zijn hoog.
- **Goede fit**: het model capteert de werkelijke structuur zonder ruis. De trainfout en testfout zijn beide laag en vergelijkbaar.
- **Overfitting**: het model is te complex en heeft de traindata "gememoriseerd", inclusief de ruis. De trainfout is laag maar de testfout is hoog.

Voorbeeld: polynomiale regressie

Beschouw een dataset van staalsterkte als functie van koolstofgehalte. Een polynoom van graad 1 (een rechte lijn) kan te weinig zijn als de relatie kromgelijnd is (underfitting). Een polynoom van graad 15 zal de vijftien datapunten perfect doorgaan maar belachelijke voorspellingen geven voor nieuwe data (overfitting). Een polynoom van graad 2 of 3 is vermoedelijk de juiste keuze.

2.6 Bias-variance trade-off

We willen nu theoretisch beter begrijpen waarom de prestatie op een testset de typische **U-vormige curve** volgt en wat het betekent dat het model overfit of underfit. Daarom voeren we de **bias-variance** decompositie van de totale fout in.

Theoretisch

Om uit te leggen wat dit betekent moeten we een paar noties invoeren:

- We herinneren eerst aan de notie van de regressielijn als voorwaardelijke verwachtingswaarde $f(x) = \mathbf{E}[Y \mid X = x]$.
- Daarnaast noteren we met $\hat{f}(x; D)$ de waarde die ons model, getraind op de dataset D zou voorspellen voor het punt x .
- Een nieuw idee bestaat erin dat we in plaats van één model te trainen in gedachten *steeds nieuwe modellen* trainen op steeds nieuwe datasets en de gemiddelde voorspelling nemen. Dit noteren we met $\tilde{f}(x) = \mathbf{E}_D[\hat{f}(x, D)]$. Dit is een theoretisch concept, gebaseerd op oneindig veel data!

Men kan dan aantonen dat de verwachte fout van het model $\hat{f}$, getraind op de data D , voor gegeven x kan worden ontbonden als:

$$\mathbf{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}[\hat{f}(x)]^2}_{\text{systematische fout}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{gevoeligheid}} + \underbrace{\sigma^2}_{\text{irreducibele ruis}}$$

De verwachtingswaarde $\mathbf{E}$ loopt hier over alle mogelijke datasets die we hadden kunnen trekken en alle mogelijke y die we hadden kunnen observeren bij de gegeven x .

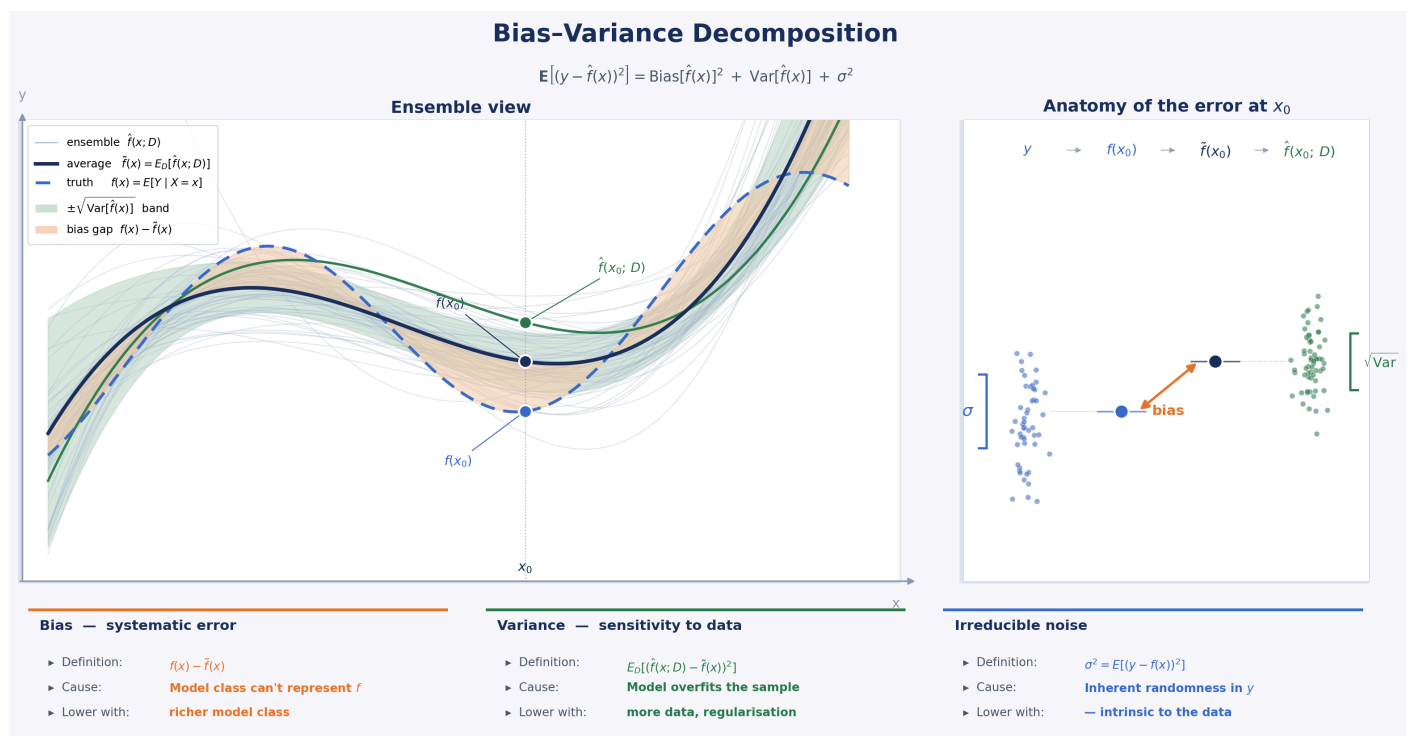
Bias: We verwachten intuïtief dat $\tilde{f}$ een zeer goede benadering zal zijn voor de werkelijke f . Dat dit toch niet het geval schrijven we toe aan het onvermogen van onze modelklasse om f werkelijk voor te stellen. De fout die overblijft is de *bias*: $f(x) - \tilde{f}(x)$.

Variance: Onze berekende $\hat{f}(x; D)$ is een inschatting van de $\tilde{f}(x)$ die we zouden kunnen berekenen met oneindig veel datasets: het is in feite een sample uit de collectie van alle mogelijke $\hat{f}(x; D)$, over alle mogelijke D . De kwadratische afwijking geeft aan hoe typisch of atypisch onze specifieke $\hat{f}(x; D)$ is, en nemen we het gemiddelde over alle datasets D waarvan we hadden kunnen vertrekken, dan verkrijgen we de *variantie*: $\mathbf{E}[(\tilde{f}(x) - \hat{f}(x; D))^2]$.

Irreduciebele fout: Tot slot zal zelfs kennis van de exacte regressielijn $f(x) = \mathbf{E}[Y|X = x]$ niet volstaan om de y die bij een gegeven x hoort exact te voorspellen ten gevolge van de intrinsieke willekeur. De overgebleven fout is intrinsiek aan de data en heeft niks te maken met de dataset die we hebben gekregen. Dit is de *irreduciebele fout*: $\mathbf{E}[(y - f(x))^2]$.

Bias-variance decompositie zegt dus dat de verwachte model-fout altijd een combinatie is van deze drie termen:

$$y \xrightarrow{\sigma^2} f(x) \xrightarrow{\text{bias}} \tilde{f}(x) \xrightarrow{\text{variance}} \hat{f}(x; D)$$

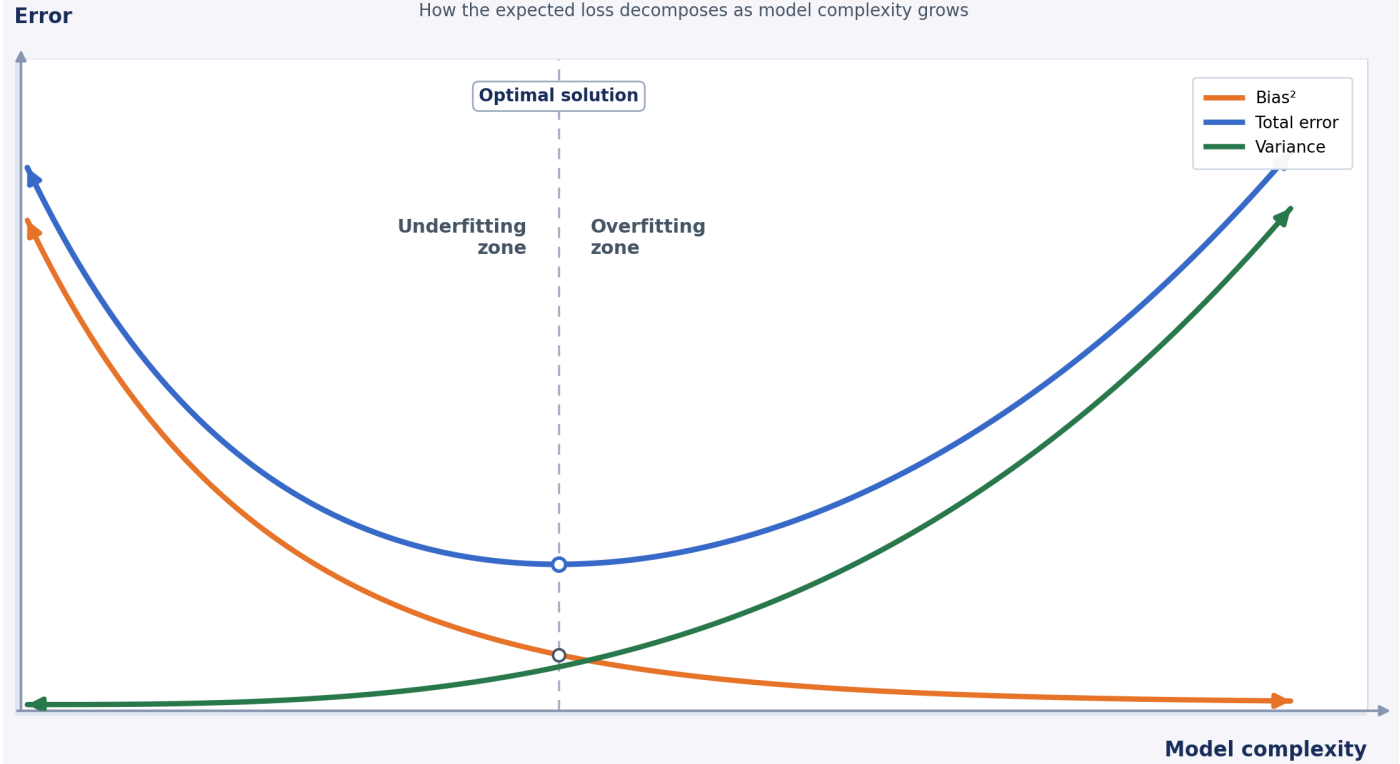


Decompositie van de totale fout in drie termen: de bias (kwadraat), variance, en irreduciebele fout.

Bias-variance trade-off is de vaststelling dat wanneer je de bias-term probeert te verkleinen met een andere keuze van hyperparameters, je vaak zal zien dat de variance daardoor groter wordt, en vice versa. Het ideale punt zal dus de best mogelijke afweging moeten maken tussen bias en variance.

The Bias-Variance Trade-off

How the expected loss decomposes as model complexity grows



De bias-variance curve: naarmate de modelcomplexiteit toeneemt, daalt de bias maar stijgt de variance. Er bestaat een optimale complexiteit die de totale fout minimaliseert.

Meer intuïtief

Bias meet de *systematische afwijking*: hoe ver ligt de gemiddelde voorspelling van het model af van de ware waarde? Hoge bias wijst op een te eenvoudig model dat de werkelijkheid niet goed kan vatten (*underfitting*).

Variance meet de *gevoeligheid voor toevallige fluctuaties* in de traindata: hoe sterk varieert de voorspelling van het model naargelang de traindata? Hoge variance wijst op een te complex model dat de ruis in de traindata als patroon interpreteert (*overfitting*).

Irreducibele ruis σ^2 is de inherente variabiliteit in de data zelf — die kan geen enkel model elimineren.

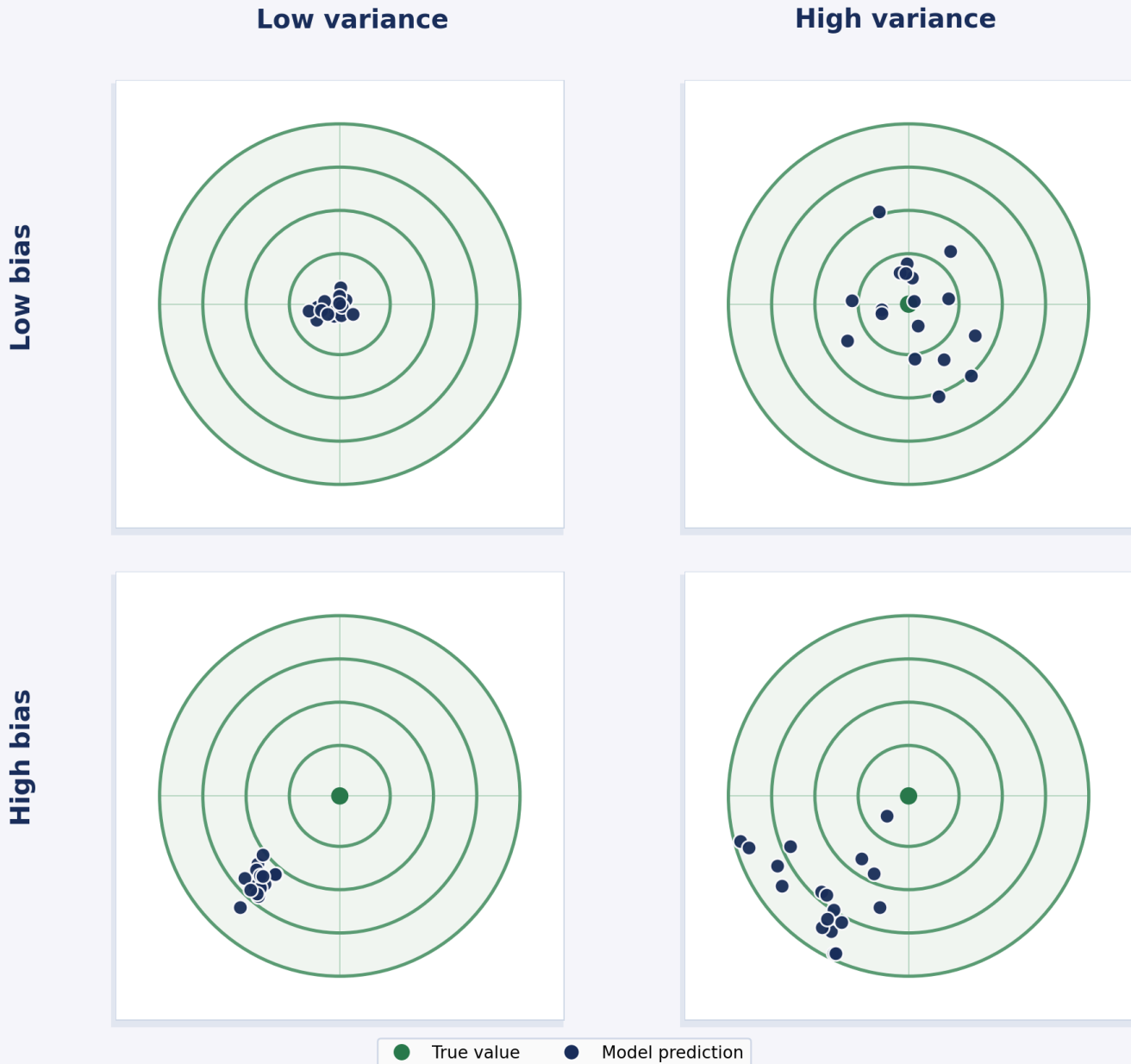
💡 Analogie met procesbeheersing

De bias-variance afweging is de statistische tegenhanger van het klassieke onderscheid tussen **nauwkeurigheid** (*accuracy*, ofwel bias) en **precisie** (*precision*, ofwel variance) in SPC (statistical process control).

- Een model met hoge bias en lage variance: consistent, maar systematisch verkeerd.
- Een model met lage bias en hoge variance: gemiddeld correct, maar onbetrouwbaar van meting tot meting.

Bias vs. Variance — a visual analogy

Each target shows where a model's predictions land relative to the truth



Visualisatie van de bias-variance afweging.

2.7 Confusion matrices

Evaluatie van classificatiemodellen

Bij regressieproblemen is de fout relatief rechtlijnig te meten (MSE, MAE). Bij classificatieproblemen liggen de zaken complexer: een model voorspelt een *klasse* in plaats van een getal, en er zijn meerdere manieren waarop een voorspelling fout kan zijn.

De **confusion matrix** brengt de volledige prestatie van een classificatiemodel in kaart door te tellen hoe vaak elke combinatie van werkelijke en voorspelde klasse voorkomt.

Binaire classificatie

Voor een binair classificatieprobleem (twee klassen: "positief" en "negatief") heeft de confusion matrix de volgende structuur:

	Voorspeld: Positief	Voorspeld: Negatief
Werkelijk: Positief	True Positive (TP)	False Negative (FN)
Werkelijk: Negatief	False Positive (FP)	True Negative (TN)



Voorbeeld van een confusion matrix met afgeleide metrics.

De vier cellen leiden tot een reeks nuttige metriek:

- **Nauwkeurigheid** (*accuracy*): $\frac{TP+TN}{TP+TN+FP+FN}$ — het aandeel correct geclassificeerde observaties. Misleidend bij sterk onevenwichtige klassen.
- **Precisie** (*precision*): $\frac{TP}{TP+FP}$ — van alle positieve voorspellingen, hoeveel zijn er werkelijk positief? Relevant wanneer valse posities kostbaar zijn.
- **Recall** (ook *sensitiviteit* of *TPR*): $\frac{TP}{TP+FN}$ — van alle werkelijk positieve gevallen, hoeveel werden correct gedetecteerd? Relevant wanneer valse negatieven kostbaar zijn.

- **F1-score:** $\frac{2 \cdot \text{precisie} \cdot \text{recall}}{\text{precisie} + \text{recall}}$ — het harmonisch gemiddelde van precisie en recall; nuttig als evenwichtsmaatstaf.

! Te onthouden

De keuze van de evaluatiemetriek is een **domeinbeslissing**, geen technische. Ze hangt af van de relatieve ernst van de twee soorten fouten:

- In kwaliteitscontrole waarbij defecten de veiligheid raken: *recall* is cruciaal — een gemist defect is erger dan een onterechte afkeur.
- In een alarmsysteem waarbij vals alarm operationeel duur is: *precisie* is crucialer — nutteloze alarmen eisen resources.

i Meerklassen-classificatie

De confusion matrix generaliseert vanzelf naar meerdere klassen: een $K \times K$ matrix waarbij rij i de werkelijke klasse aangeeft en kolom j de voorspelde klasse. De diagonaalcellen zijn de correcte voorspellingen; alles daarbuiten zijn fouten. Metriek zoals precisie en recall worden dan per klasse berekend (en eventueel gemiddeld).

3 Overzicht van methoden

Nu we de basisbegrippen verteerd hebben zijn we klaar voor een overzicht van de meest gebruikte methoden. Het doel is niet om volledig te zijn maar die methoden aan te stippen waarvan ik meen, uit persoonlijke ondervinding, dat je er wel eens mee in aanraking zou kunnen komen.

We zullen zowel methoden zien uit de wereld van het *supervised learning* als het *unsupervised learning*

💡 Geen silver bullet

Er bestaat geen universeel beste methode. De keuze hangt af van:

- De omvang en dimensionaliteit van de data
- De aanwezigheid van labels
- De aard van de relatie (lineair vs. niet-lineair, glad vs. discontinu)
- De vereiste interpreteerbaarheid
- Rekenkundige en tijdsbeperkingen

In de volgende les oefenen we met de meest gangbare methoden in Python.

3.1 Supervised learning

Lineaire regressie en aanverwanten

Lineaire regressie is de meest fundamentele supervised learning-methode en de historische basis van modern machine learning. Het model veronderstelt een lineaire relatie:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

"Lineair" verwijst hier naar lineariteit in de coëfficiënten β_i , niet noodzakelijk in de invoervariabelen. Een polynomiale regressie $y = \beta_0 + \beta_1 x + \beta_2 x^2$ is ook lineair in deze zin.

Lineaire regressie minimaliseert de MSE als loss-functie en heeft een analytische oplossing: de **Ordinary Least Squares (OLS)**-schatte. De coëfficiënten zijn interpreteerbaar en er bestaan geijkte statistische tests voor significantie.

Uitbreidingen van lineaire regressie

Methode	Kernidee
Ridge-regressie	Voegt een L_2 -bestrafing toe op de grootte van coëfficiënten. Nuttig bij multicollineariteit en hoog-dimensionale data.
Lasso-regressie	Voegt een L_1 -bestrafing toe. Forceert sommige coëfficiënten naar nul $\rightarrow$ <i>ingebouwde variabelenselectie</i> .
Elastic Net	Combinatie van Ridge en Lasso.
Quantielregressie	Modelleert mediaan of andere kwantielen in plaats van het gemiddelde. Robuuster bij uitschieters.
Total Least Squares / Deming-regressie	Houdt rekening met meetfout in zowel x als y (zie ook het staalkoeling-voorbeeld uit lecture 1).
Gegeneraliseerde lineaire modellen (GLM)	Breidt lineaire regressie uit naar niet-Gaussische uitkomsten (lognormaal, Poisson, binomiaal, ...).

Neurale netwerken

Neurale netwerken zijn essentieel een elegante manier om heel complexe functies op te bouwen uit eenvoudige. Een neurale netwerk is een stelsel van lagen artificiële neuronen die samen een complexe, niet-lineaire transformatie tussen input en output modelleren.

Bouwsteen: het enkelvoudig neuron

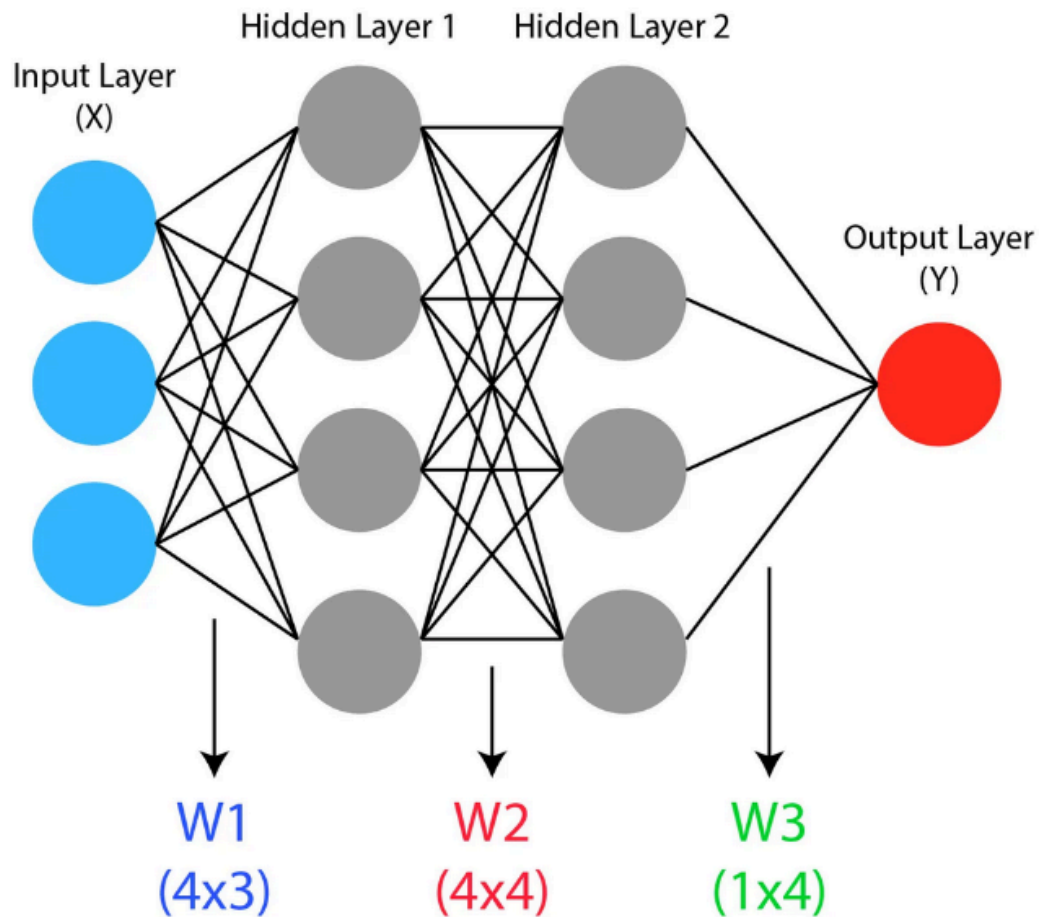
Een enkel neuron berekent:

$$y = f(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b)$$

waarbij w_i de **gewichten** zijn, b een **bias**-term, en f een niet-lineaire **activatiefunctie**. Veel gebruikte activatiefuncties zijn de ReLU ($f(x) = \max(0, x)$), de sigmoid en de tanh.

Architectuur

Door neuronen in lagen te stapelen ontstaan zogenaamde **verborgen lagen** (*hidden layers*):



$$Y = f(W3 * f(W2 * f(W1 * X + B1) + B2) + B3)$$

The equation is annotated with boxes and arrows to show the flow of information through the layers:

- A box labeled **Input Layer** encloses the input X .
- A box labeled **Hidden Layer 1** encloses the sub-expression $f(W1 * X + B1)$.
- A box labeled **Hidden Layer 2** encloses the entire sub-expression $f(W2 * f(W1 * X + B1) + B2)$.
- The final output Y is produced by the function $f(W3 * \dots + B3)$.

Een neurale netwerk met invoerlaag, verborgen lagen en uitvoerlaag.

- **Invoerlaag:** neemt de kenmerken van de data als invoer.
- **Verborgen lagen:** leren interne representaties. Het aantal lagen en neuronen is een hyperparameter.
- **Uitvoerlaag:** produceert de voorspelling (getal bij regressie, kansen bij classificatie).

Deep learning verwijst naar netwerken met meerdere verborgen lagen. De kracht ligt in de **universele approximatiestelling**: een neurale netwerk met voldoende neuronen kan elke continue functie arbitrair goed benaderen.

Hoe leert een neuraal netwerk?

Het trainingsproces verloopt via **gradient descent**:

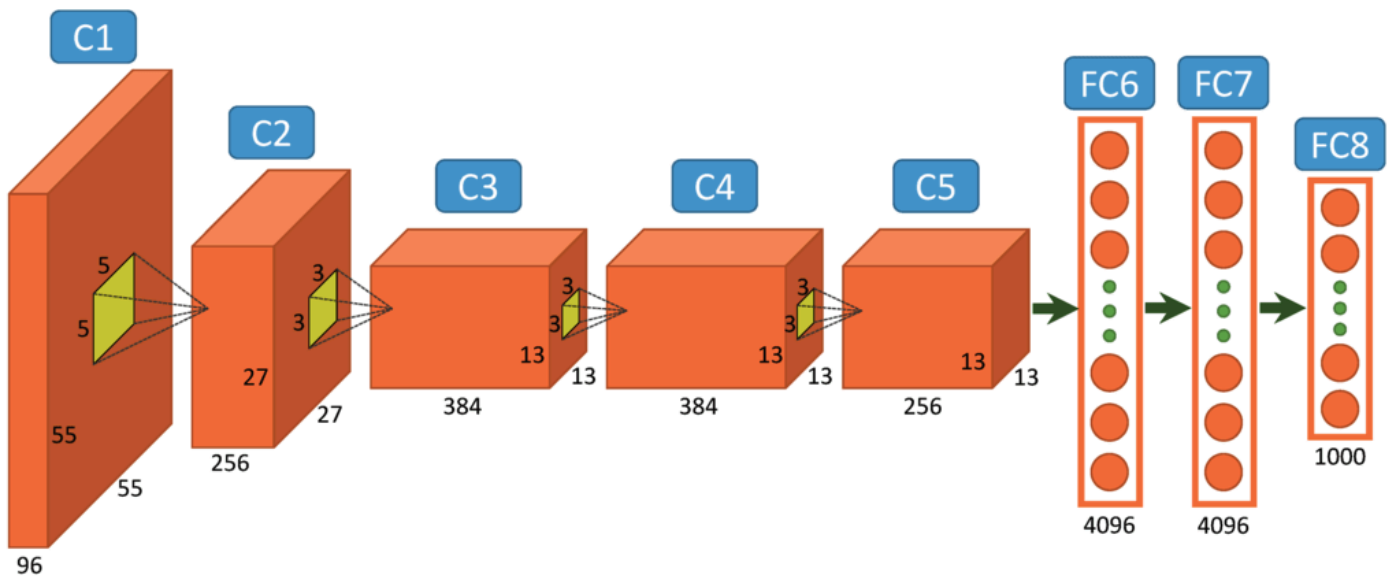
- 1. Initialiseer de gewichten willekeurig.
- 2. Bereken de loss op een batch traindata.
- 3. Bereken de gradiënt van de loss ten opzichte van de gewichten via **backpropagation**.
- 4. Pas de gewichten aan in de richting die de loss verlaagt.
- 5. Herhaal tot convergentie.

⚠ Neurale netwerken en overfitting

Neurale netwerken zijn bijzonder vatbaar voor overfitting vanwege hun grote aantal parameters (van duizenden tot miljarden). Regularisatietechnieken zoals *dropout*, *early stopping* en *weight decay* zijn onmisbaar. Cross-validatie is essentieel.

Varianten

Architectuur	Toepassing
Feedforward network (MLP)	Algemene regressie en classificatie op tabeldata
Convolutioneel netwerk (CNN)	Beeldherkenning, signaalverwerking
Recurrent network (RNN/LSTM)	Tijdreeksen, sequentiële data
Transformer	Taal (LLMs), tijdreeksen, steeds vaker ook beelden
Autoencoder	Anomaliedetectie, dimensiereductie, compressie



AlexNet: de CNN-architectuur die in 2012 de ImageNet-compitie won en het tijdperk van deep learning inluidde. Er zijn ong. 62 miljoen parameters.

💡 Wanneer neurale netwerken?

Neurale netwerken schitteren wanneer:

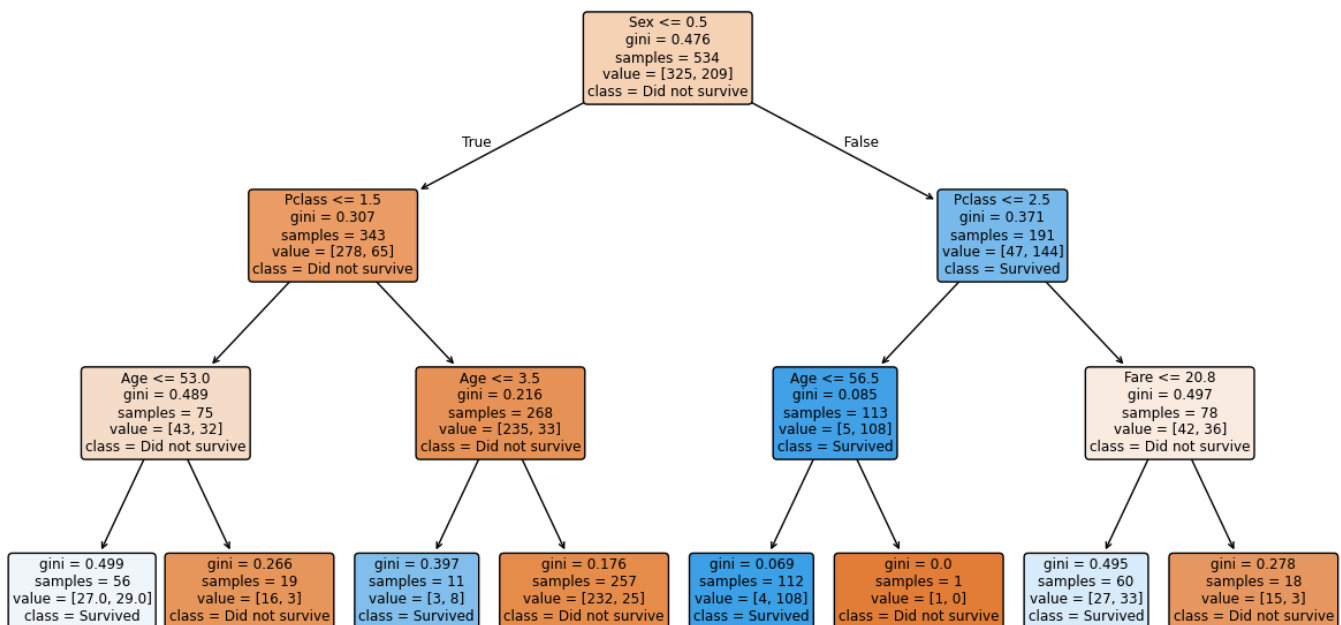
- De data omvangrijk is (duizenden tot miljoenen observaties).
- De relatie sterk niet-lineair en moeilijk te formaliseren is.
- De invoer ongestructureerde data is (beelden, tekst, audio).

Ze zijn minder geschikt voor:

- Kleine datasets (< een paar honderd observaties): overfitting ligt op de loer.
- Situaties waar interpreteerbaarheid vereist is: neurale netwerken zijn "zwarte dozen".
- Productieomgevingen met strenge rekenkundige beperkingen.

Boom-gebaseerde methoden

Beslissingsbomen (*decision trees*) classificeren of voorspellen door de invoerruimte successievelijk op te splitsen aan de hand van drempelwaarden op de kenmerken. Elke interne knoop stelt een vraag ("Is koolstofgehalte > 0,3%?"), elk blad stelt een voorspelling.



Voorbeeld van een beslissingsboom.

Voordelen: eenvoudig te visualiseren en te interpreteren, minimale data-voorbereiding vereist. Nadeel: gevoelig voor overfitting en de splitsingsdrempels missen vaak fysische betekenis.

Ensemble-methoden combineren meerdere beslissingsbomen om de prestatie te verbeteren:

Ensemble-methoden op basis van bomen

Methode	Kernidee	Sterkte
Random Forest	Traint k bomen op willekeurige steekproeven van data en kenmerken; middeelt de voorspellingen	Stabiel, robuust tegen overfitting, weinig hyperparameters

Methode	Kernidee	Sterkte
Gradient Boosting (XGBoost, LightGBM)	Traint opeenvolgende bomen waarbij elke boom de fout van de vorige corrigeert	Zeer hoge prestatie op tabeldata; winnaar van veel Kaggle-competities
AdaBoost	Herwaardeert moeilijke observaties bij elke iteratie	Historisch; minder populair dan gradient boosting

Gradient boosting-methoden zoals XGBoost en LightGBM zijn momenteel de **state of the art voor gestructureerde tabeldata** — het type data dat het meest voorkomt in industriële toepassingen.

💡 Boom-methoden in de industrie

Voor kwaliteitscontrole, procesoptimalisatie en voorspellend onderhoud op tabeldata (sensor-uitlezingen, procesparameters) zijn gradient boosting-methoden vaak de eerste keuze:

- Ze vereisen weinig data-voorbereiding (geen normalisatie nodig).
- Ze handelen ontbrekende waarden intern af.
- Ze zijn minder gevoelig voor irrelevante kenmerken dan lineaire methoden.
- Ze leveren ingebouwde **feature importance**-scores die aangeven welke kenmerken het meest bijdragen aan de voorspelling.

3.2 Unsupervised learning

Bij unsupervised learning zijn er geen labels: het model krijgt enkel de invoerdata x_i en probeert daar zelfstandig structuur in te ontdekken. Dit is bijzonder nuttig voor verkenning van nieuwe datasets, anomaliedetectie en data-compressie.

Clustering

Clusteringmethoden verdelen de data in groepen (*clusters*) zodat observaties binnen een cluster meer op elkaar lijken dan observaties in verschillende clusters.

K-means is de meest gekende methode. Het algoritme kiest k centroides en wijst elke observatie toe aan de dichtstbijzijnde centroïde. Iteratief worden de centroides verplaatst tot convergentie. Nadeel: het aantal clusters k moet vooraf worden opgegeven.

Hiërarchische clustering bouwt een boomstructuur (*dendrogram*) door clusters stapsgewijs samen te voegen (agglomeratief) of te splitsen (divisief). Voordeel: men hoeft k niet op voorhand te kiezen.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) definieert clusters als dichte gebieden in de data, gescheiden door gebieden met lage dichtheid. Robuust voor uitschieters en kan clusters van willekeurige vorm detecteren.

📌 Toepassingen van clustering in de industrie

- **Klantsegmentatie:** klanten groeperen op basis van aankoopgedrag om gerichte acties te ondernemen.

- **Producttypering:** producten of coils groeperen op basis van procesparameters, zonder vooraf gedefinieerde categorieën.
- **Anomaliedetectie** (indirect): een observatie die niet past in geen enkel cluster kan een anomalie zijn.

Dimensiereductie

Hoog-dimensionale data is moeilijk te visualiseren en te verwerken. Dimensiereductie-methoden projecteren de data op een ruimte met minder dimensies, terwijl zoveel mogelijk informatie behouden blijft.

Principal Component Analysis (PCA) is de klassieke lineaire methode. PCA vindt de richtingen in de hoog-dimensionale ruimte waarlangs de data het meest varieert (*hoofdcomponenten*) en projecteert de data op die richtingen. De eerste hoofdcomponent verklaart de meeste variantie, de tweede de meeste overblijvende variantie, enzovoort.

PCA is nuttig voor:

- Visualisatie van hoog-dimensionale data in 2D of 3D.
- Ruisonderdrukking (de componenten met weinig variantie bevatten vaak ruis).
- Preprocessing voor andere methoden om de vloek van dimensionaliteit te beperken.

t-SNE (t-distributed Stochastic Neighbor Embedding) en **UMAP** (Uniform Manifold Approximation and Projection) zijn niet-lineaire methoden die bijzonder geschikt zijn voor visualisatie. Ze bewaren de lokale structuur (nabuurenschappen) beter dan PCA bij sterk niet-lineaire data.

Anomaliedetectie

Anomaliedetectie (ook *outlier detection*) identificeert observaties die sterk afwijken van het normale gedrag van de data. Dit is bijzonder relevant voor:

- **Voorspellend onderhoud:** tijdig detecteren van afwijkend machinegedrag vóór een defect optreedt.
- **Kwaliteitscontrole:** detecteren van procesafwijkingen of meet-fouten.
- **Fraudedetectie:** ongewone transactiepatronen opsporen.

Gangbare methoden:

- **Isolation Forest:** isoleert anomalieën via willekeurige partitionering — abnormale punten zijn makkelijker te isoleren dan normale.
- **Autoencoder:** een neurale netwerk dat data comprimeert en reconstrueert. Anomalieën worden slecht gereconstrueerd (hoge reconstructiefout).
- **One-class SVM:** leert de grens van het “normale” gebied in de invoerruimte.

Anomaliedetectie zonder labels

In veel industriële situaties zijn er weinig of geen gelabelde anomalieën beschikbaar — defecten zijn per definitie zeldzaam. Unsupervised anomaliedetectie is dan de enige optie: het model leert wat “normaal” is en signaleert afwijkingen zonder voorbeelden van anomalieën gezien te hebben.

3.3 Bayesiaanse methoden

Een andere filosofie

De tot nu toe besproken methoden zijn overwegend **frequentistisch**: enkel de data is willekeurig; de modelparameters zijn in principe niet-stochastische grootheden. Het heeft dus geen zin om een uitspraak te doen over de "kans" dat een parameter in een bepaald gebied ligt!

De Bayesiaanse filosofie neemt een fundamenteel ander standpunt in: **parameters zijn zelf stochastische grootheden**. Onzekerheid over de waarde van een parameter is een vorm van subjectieve onzekerheid over hoe de natuur werkelijk in elkaar steekt. Een groot voordeel is dat parameters en data daardoor op een gelijkaardige manier kunnen worden behandeld; een groot nadeel dan weer is dat de subjectiviteit van deze onzekerheid tot grote discussie aanleiding kan geven.

Het vertrekpunt is de **regel van Bayes**:

$$P(\theta \mid \text{data}) = \frac{P(\text{data} \mid \theta) \cdot P(\theta)}{P(\text{data})}$$

- $P(\theta)$: de **prior** — onze kennis of aannames over de parameters *vóór* het zien van de data. Dit is waar de subjectiviteit het gebied binnenkomt.
- $P(\text{data} \mid \theta)$: de **likelihood** — hoe waarschijnlijk is de waargenomen data gegeven bepaalde parameterwaarden? Men noemt dit soms *het data-genererend model* noemt: gegeven een keuze van parameters, hoe zouden we dan verwachten dat de data eruit ziet?
- $P(\theta \mid \text{data})$: de **posterior** — onze bijgewerkte kennis over de parameters *na* het zien van de data.

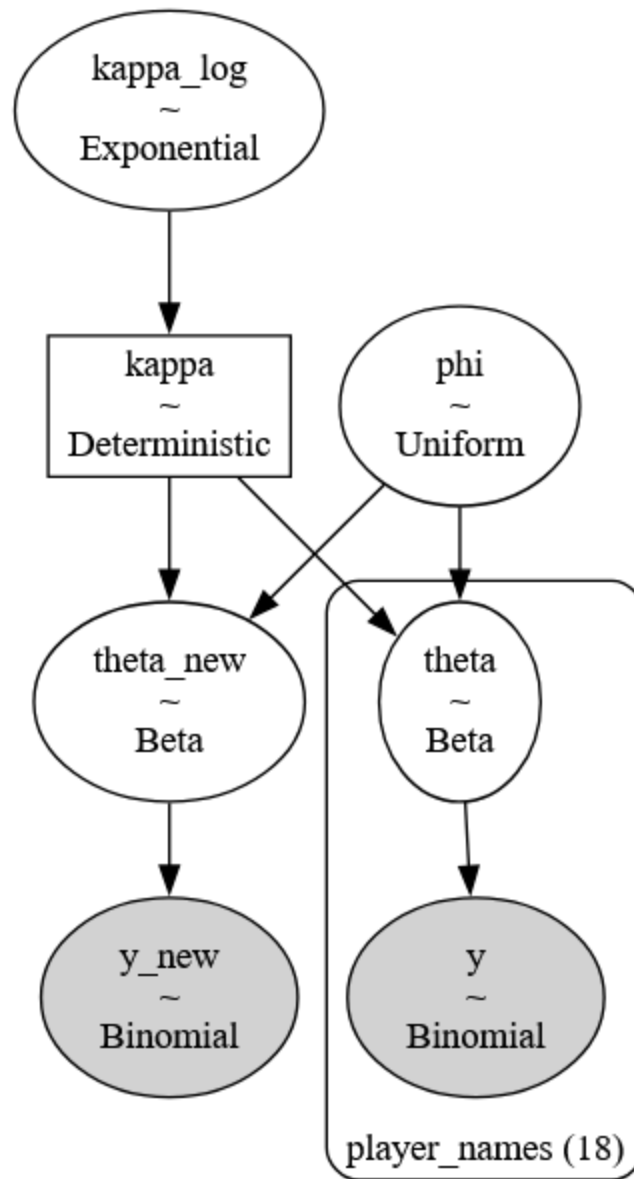
⚠ Hot take

Bayesiaanse methoden zijn bijzonder onderschat voor de industrie. Ze zijn bijzonder geschikt om bepaalde proceskennis in een model onder te brengen.

Daar waar je in wetenschappelijk onderzoek lange discussies kan voeren over de vraag of het aanvaardbaar is om *a priori* al te stellen dat de leeftijd van het heelal grootte-orde enkele miljarden jaren is, is het in een staalfabriek bijna altijd aanvaardbaar om te stellen dat de temperatuur van vloeibaar staal tussen de 1400 en 1800 °C zal zijn. Vaak hebben we voorkennis die nog veel preciezer is en door die niet mee te nemen in een model is onze schatting niet optimaal.

Bayesiaanse netwerken

Een **Bayesiaans netwerk** is een probabilistisch grafisch model dat de gezamenlijke kansverdeling van een stelsel variabelen beschrijft via conditionele afhankelijkheden. Het netwerk bestaat uit knopen (variabelen) en pijlen (causale of statistische relaties).



Voorbeeld van een Bayesiaans netwerk.

In tegenstelling tot de meeste machine learning-methoden:

- Modelleert een Bayesiaans netwerk de **volledige kansverdeling** over de variabelen, niet slechts een regressielijn.
- Kan het voorafgaande **domeinkennis** (de prior) opnemen — bijzonder waardevol bij kleine datasets.
- Geeft het **onzekerheidsintervallen** mee bij elke voorspelling.

💡 Wanneer Bayesiaanse methoden?

Industriële productiedata verschilt fundamenteel van de "big data" van technologiebedrijven:

- Minder observaties (geen miljoenen gebruikers, maar honderden of duizenden coils, batches of producten).
- Rijkere domeinkennis: ingenieurs kennen de fysica van het proces.
- De structuur van het probleem is vaak bekend: welke variabelen beïnvloeden welke andere.

In die context zijn Bayesiaanse methoden bijzonder waardevol:

- Ze schalen niet goed naar gigantische datasets, maar voor middelgrote datasets zijn ze concurrentieel.
- Ze laten toe om kennis uit eerdere experimenten of domeinexpertise in het model te verankeren.
- Ze geven een inschatting van onzekerheid; iets wat klassieke methoden vaak niet doen.

Voorbeeld: coefficient pooling

Stel dat we drie productielijnen hebben met dezelfde fysische wetmatigheid maar met elk een eigen offset (bv. door kalibratieverband). In klassieke regressie zou men drie aparte modellen fitten. In een Bayesiaans netwerk kan men de gedeelde parameter (de gemeenschappelijke coëfficiënt) uitdrukken als een gedeeld “knooppunt” in het netwerk, waardoor de schattingen van alle drie de lijnen elkaar versterken — zelfs als elk afzonderlijk weinig data heeft.

Praktische implementatie

Bayesiaanse inferentie vereist het berekenen van complexe integralen die analytisch zelden oplosbaar zijn. In de praktijk worden algoritmen gebruikt die *steekproeven trekken* uit de posterior-verdeling, zoals **Markov Chain Monte Carlo (MCMC)**. Bibliotheken als **PyMC** en **Stan** maken dit toegankelijk.

4 Studiewijzer

Na deze les bent u vertrouwd met:

Kernbegrippen

- **Terminologie** zoals AI, ML, Supervised en Unsupervised Learning kunnen plaatsen
- **Loss-functies**: de rol van lossfuncties in ML begrijpen, bekend zijn met MSE en MAE.
- **Cross-validatie** en het belang van train-test-split kunnen uitleggen
- **Bias-variance**: de trade-off tussen bias en variance ifv model complexiteit kunnen uitleggen
- **Confusion matrices** en de evaluatie van classificatiemodellen

Methoden

- **Lineaire regressie**: weten dat er uitbreidingen zijn zoals logistical en quantile regression
- **Neurale netwerken**: notie hebben van *hidden layers* en *deep learning*
- **Beslissingsbomen**: algemeen werkingsprincipe kunnen uitleggen
- **Bayesiaanse methoden**: bekend zijn met voordelen voor gestructureerde data

4.1 Literatuur

An Introduction to Statistical Learning — James, Witten, Hastie & Tibshirani (2021, 2e editie)

Een toegankelijk maar grondig leerboek voor supervised learning met toepassingen in R en Python. Vrij beschikbaar via de website van de auteurs. Bijzonder aanbevolen voor wie de theoretische achtergrond van de behandelde methoden beter wil begrijpen.

Hands-On Machine Learning with Scikit-Learn, and PyTorch — Aurélien Géron (3e editie, 2022)

Een praktisch georiënteerd handboek met concrete voorbeelden in Python.

Probabilistic Machine Learning — Kevin P. Murphy (1e editie, 2022)

Beschikbaar via de [website van de auteur](#)